

將單體式網站遷移至 Google Kubernetes Engine 上的微服務

來源：<https://codelabs.developers.google.com/codelabs/cloud-monolith-to-microservices-gke?hl=zh-tw>

步驟數：10

本研究室將逐步引導您將單體式網站拆解成微服務，並部署至 Google Kubernetes Engine。

目錄

1. [簡介](#)
2. [環境設定](#)
3. [複製來源存放區](#)
4. [建立 GKE 叢集](#)
5. [Deploy Existing Monolith](#)
6. [將訂單遷移至微服務](#)
7. [將產品遷移至微服務](#)
8. [將前端遷移至微服務](#)
9. [清除所用資源](#)
10. [恭喜！](#)

1. 簡介

為何要從單體式應用程式遷移至微服務架構？將應用程式拆解成微服務具有下列優點；其中大多數的優點都是源自於微服務是鬆散連結的事實。

- 微服務可以單獨測試及部署。部署單元越小，部署起來越容易。
- 微服務可以用不同的語言及架構實作。每個微服務可以分別採用最適合其特定用途的技術。
- 微服務可以由不同的團隊管理。微服務之間的界限分明，因此較容易由專責團隊分別管理。
- 遷移到微服務後，即可降低各團隊之間的相依性。微服務的 API 專責團隊只需負責管理，無須考慮微服務如何實作及發布週期等問題。
- 設計容錯機制更容易。由於服務之間的界限分明，所以當服務中斷時，決定該怎麼處理會比較容易。

與單體架構相比，微服務的缺點包括：

- 由於以微服務為基礎的應用程式是由不同服務組成的網路，而這些服務之間的互動通常不怎麼明顯，因此可能會增加系統的整體複雜性。
- 與單體的內部架構不同，微服務需透過網路進行通訊。在某些情況下，這可能會存在安全顧慮。[Istio](#) 透過自動加密微服務之間的流量來解決這個問題。
- 由於服務之間的延遲問題，微服務的效能很難達到單體式應用程式的水準。
- 系統行為不是單一服務造成的，而是由許多服務和它們之間的互動所造成。因此，要瞭解系統在實際工作環境中的表現 (即系統的「可觀察性」) 會比較困難。[Istio](#) 也能解決此問題。

在本實驗室中，我們會在 Google Kubernetes Engine (GKE) 中執行微服務。Kubernetes 是一個代管、託管、調度及部署容器的平台。容器是一種打包和執行程式碼的可攜性技術，為微服務提供了理想的條件，使每個微服務都可以執行自己的容器。

在本實驗室中，我們將現有的單體式應用程式部署至 **Google Kubernetes Engine 叢集**，然後將其拆解為微服務！

注意事項

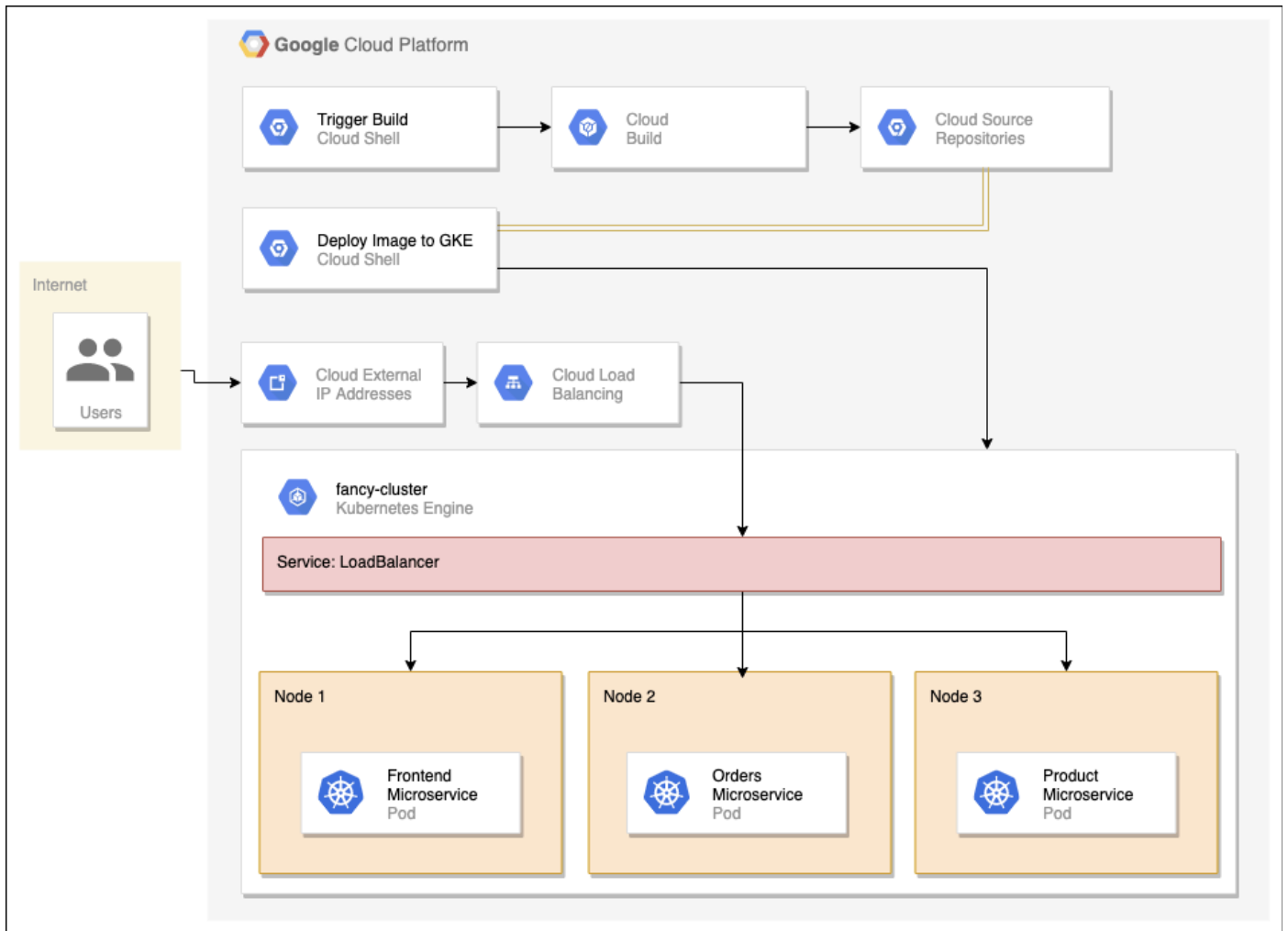
詳情請參閱下列連結：

[將單體式應用程式遷移至微服務 \(完整文章\)](#)

[微服務最佳做法](#)

微服務架構圖

首先，將單體式應用程式拆分為三項微服務，一次拆分一項。微服務包括訂單、產品和前端。我們會使用 Cloud Build 為每個微服務建立 Docker 映像檔，並從 Cloud Shell 內觸發。然後透過 Kubernetes 服務類型 LoadBalancer，在 Google Kubernetes Engine (GKE) 上部署及公開微服務。我們會為每項服務執行這項操作，同時將這些服務從單體架構中重構出來。在整個過程中，單體和微服務都會持續運作，直到最後我們才能刪除單體。



課程內容

- 如何將單體式應用程式拆解為微服務
- 如何建立 Google Kubernetes Engine 叢集
- 如何建立 Docker 映像檔
- 如何將 Docker 映像檔部署至 Kubernetes

必要條件

- 具備管理存取權的 Google Cloud Platform 帳戶，可建立專案，或具備專案擁有者角色的專案
- Docker 和 Kubernetes 的基本概念

其他資訊

[Docker](#)

[Kubernetes](#)

[將單體式應用程式遷移至微服務 \(完整文章\)](#)

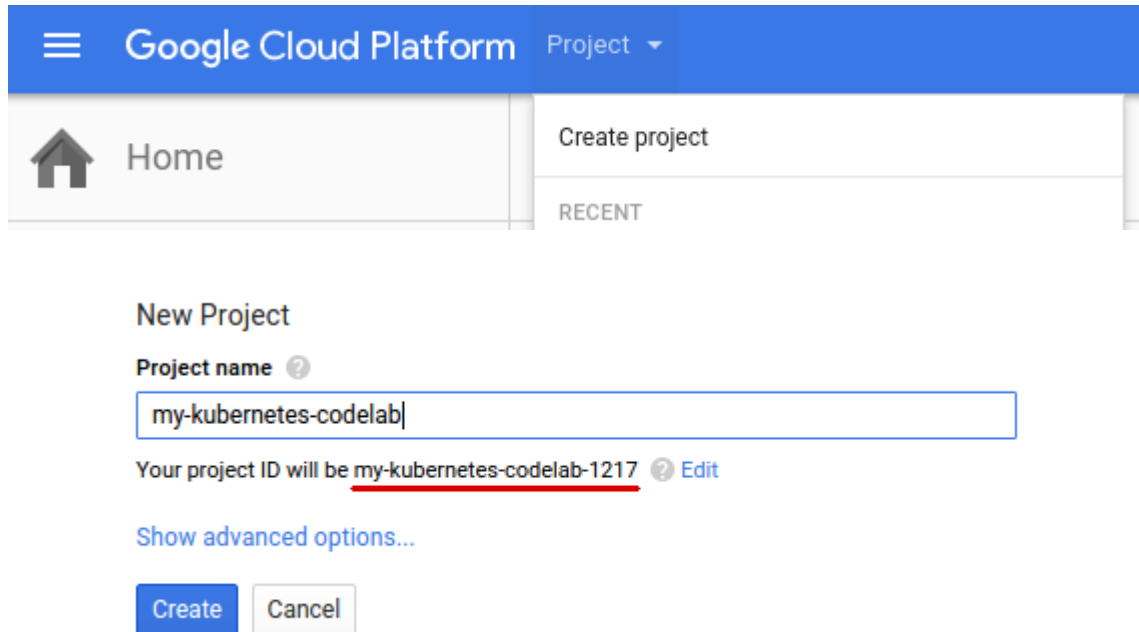
[微服務最佳做法](#)

步驟 2 · 約 5 分鐘

2. 環境設定

自修實驗室環境設定

如果您沒有 Google 帳戶 (Gmail 或 Google 應用程式)，請先[建立帳戶](#)。登入 Google Cloud Platform 主控台 (console.cloud.google.com)，然後建立新專案：



請記住專案 ID，這是所有 Google Cloud 專案中不重複的名稱 (上述名稱已遭占用，因此不適用於您，抱歉！)。本程式碼研究室稍後會將其稱為 `PROJECT_ID`。

接著，您需要在開發人員控制台中[啟用帳單](#)，才能使用 Google Cloud 資源，並[啟用 Container Engine API](#)。

完成本程式碼研究室的費用不應超過數美元，但如果您決定使用更多資源，或是將資源繼續執行 (請參閱本文件結尾的「清除」一節)，則可能會增加費用。如要瞭解 Google Kubernetes Engine 定價，請參閱[這篇文章](#)。

Google Cloud Platform 新使用者享有價值 **\$300 美元的免費試用期**。

Google Cloud Shell

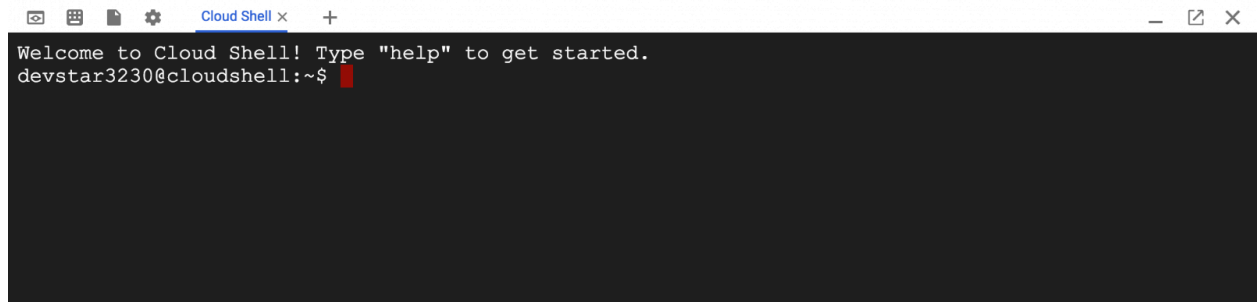
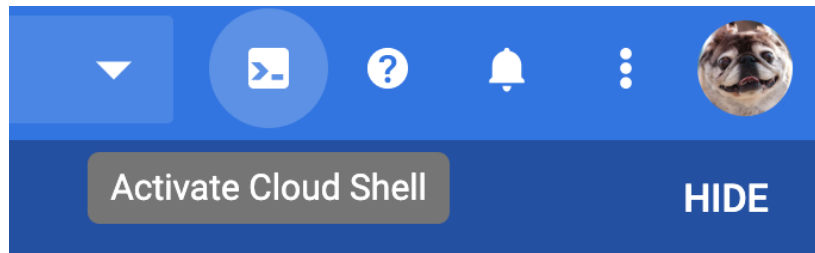
雖然可以透過筆電遠端操作 Google Cloud 和 Kubernetes，但在本程式碼研究室中，我們將使用 [Google Cloud Shell](#)，也就是在雲端執行的指令列環境。

這部以 Debian 為基礎的虛擬機器，搭載各種您需要的開發工具，並提供永久的 5GB 主目錄，而且可在 Google Cloud 運作，大幅提升網路效能並強化驗證功能。也就是說，您只需要瀏覽器 (Chromebook 也可以) 就能完成本程式碼研究室。

1. 如要從 Cloud 控制台啟用 Cloud Shell，只要按一下「啟用 Cloud Shell」



即可 (佈建並連線至環境的作業需要一些時間才能完成)。



連至 Cloud Shell 後，您應該會看到驗證已完成，專案也已設為獲派的專案 ID `PROJECT_ID`。

```
gcloud auth list
```

指令輸出

```
Credentialed accounts:
- <myaccount>@<mydomain>.com (active)
```

注意：Cloud Shell 已預先安裝 `gcloud` 指令列工具，並支援 Tab 鍵完成功能，相信您一定會喜歡。詳情請參閱 [gcloud 指令列工具總覽](#)。

```
gcloud config list project
```

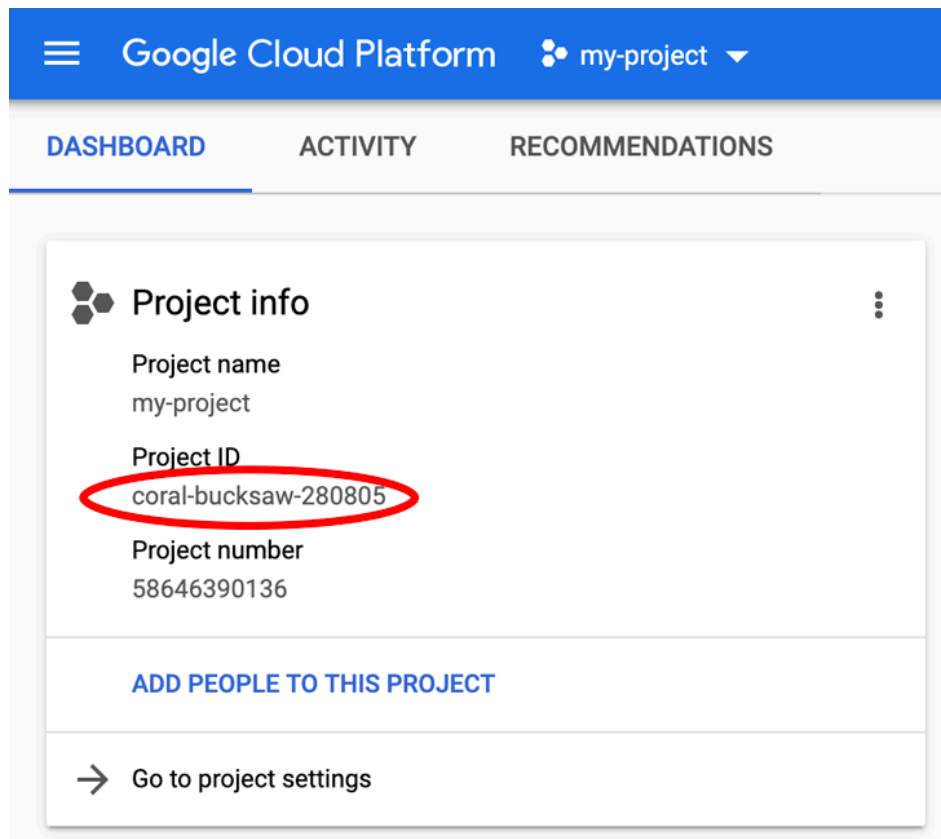
指令輸出

```
[core]
project = <PROJECT_ID>
```

如果專案未設定，請發出下列指令：

```
gcloud config set project <PROJECT_ID>
```

在尋找「`PROJECT_ID`」嗎？請檢查您在設定步驟中使用的 ID，或在 Cloud 控制台資訊主頁中尋找：



Cloud Shell 也會預設設定部分環境變數，這些變數在您執行後續指令時可能很有用。

```
echo $GOOGLE_CLOUD_PROJECT
```

指令輸出

```
<PROJECT_ID>
```

2. 最後，設定預設可用區和專案。

```
gcloud config set compute/zone us-central1-f
```

你可以選擇各種不同區域。詳情請參閱「[地區和區域](#)」。

注意：如果在自己的機器上執行 `gcloud`，各個工作階段都會保留這些設定。但在 Cloud Shell 中，每次啟動新工作階段或重新連線時，都必須重新設定。

3. 複製來源存放區

我們使用現有的單體式應用程式，也就是虛構的電子商務網站，其中包含簡單的歡迎頁面、產品頁面和訂單記錄頁面。我們只需要從 Git 存放區複製來源，就能專心將其拆解為微服務，並部署至 Google Kubernetes Engine (GKE)。

執行下列指令，將 git 存放區複製到 Cloud Shell 執行個體，並切換至適當的目錄。我們也會安裝 NodeJS 依附元件，以便在部署前測試單體應用程式。這項指令碼可能需要幾分鐘才能執行完畢。

```
cd ~  
git clone https://github.com/googlecodelabs/monolith-to-microservices.git  
cd ~/monolith-to-microservices  
./setup.sh
```

這會複製我們的 GitHub 存放區、變更為該目錄，並安裝在本機執行應用程式所需的依附元件。這項指令碼可能需要幾分鐘才能執行完畢。

步驟 4 · 約 5 分鐘

4. 建立 GKE 叢集

現在您已建立可運作的開發環境，接下來需要 Kubernetes 叢集，才能部署單體應用程式和微服務。建立叢集前，請確認已啟用適當的 API。執行下列指令，啟用 Containers API 以使用 Google Kubernetes Engine：

```
gcloud services enable container.googleapis.com
```

現在可以開始建立叢集！執行下列指令，建立名為 **fancy-cluster** 的 GKE 叢集，其中包含 **3** 個節點。

```
gcloud container clusters create fancy-cluster --num-nodes 3
```

警告

如果收到未指定區域/可用區的錯誤訊息，請參閱環境設定一節，確認已設定預設運算可用區。

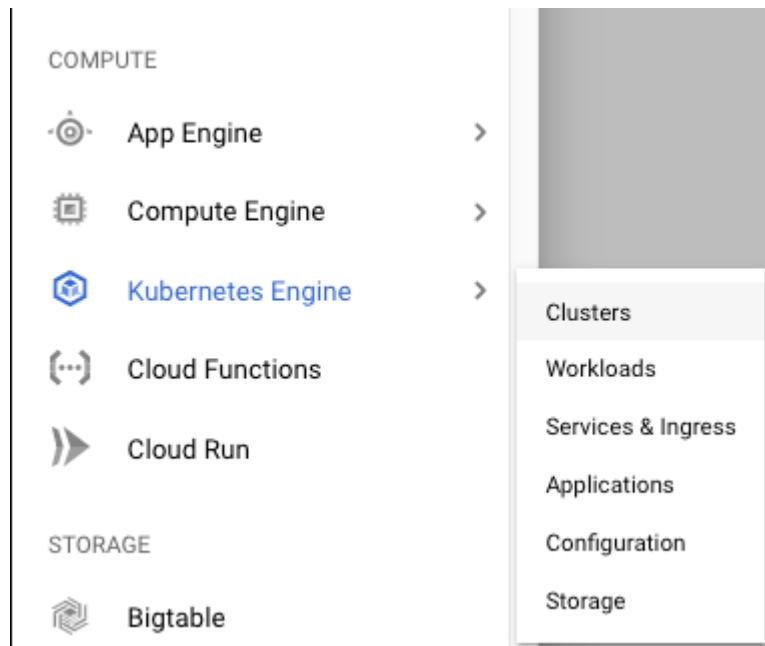
可能需要幾分鐘時間才能建立叢集。指令執行完畢後，請執行下列指令，查看叢集的三個 worker VM 執行個體：

```
gcloud compute instances list
```

輸出內容：

NAME	ZONE	MACHINE_TYPE	PREEMPTIBLE
INTERNAL_IP	EXTERNAL_IP	STATUS	
gke-fancy-cluster-default-pool-ad92506d-1ng3	us-east4-a	n1-standard-1	
10.150.0.7	XX.XX.XX.XX	RUNNING	
gke-fancy-cluster-default-pool-ad92506d-4fvq	us-east4-a	n1-standard-1	
10.150.0.5	XX.XX.XX.XX	RUNNING	
gke-fancy-cluster-default-pool-ad92506d-4zs3	us-east4-a	n1-standard-1	
10.150.0.6	XX.XX.XX.XX	RUNNING	

您也可以在 Google Cloud 控制台中查看 Kubernetes 叢集和相關資訊。按一下左上角的選單按鈕，向下捲動至 Kubernetes Engine，然後點選「叢集」。您應該會看到名為 **fancy-cluster** 的叢集。



Kubernetes clusters + CREATE CLUSTER + DEPLOY ↻ REFRESH 🗑 DELETE SHOW INFO PANEL

A Kubernetes cluster is a managed group of VM instances for running containerized applications. [Learn more](#)

Filter by label or name

☐	Name ^	Location	Cluster size	Total cores	Total memory	Notifications	Labels
☐	✅ fancy-cluster	us-east4-a	3	3 vCPUs	11.25 GB	<button>Connect</button>	

恭喜！您已建立第一個 Kubernetes 叢集！

注意事項

如果您使用現有的 Google Kubernetes Engine 叢集，或是您已透過 Google Cloud Platform Console 建立叢集，則必須執行下列指令擷取叢集憑證，並使用這些憑證設定 kubectl 指令列工具：

```
gcloud container clusters get-credentials fancy-cluster
```

如果您已用上述的 `gcloud container clusters create` 指令建立叢集，則可略過此步驟。

5. Deploy Existing Monolith

本實驗室的重點是逐步將單體式應用程式拆解為微服務，因此單體式應用程式需要先正常運作。執行下列指令碼，將單體應用程式部署至 GKE 叢集，以用於本實驗室：

```
cd ~/monolith-to-microservices
./deploy-monolith.sh
```

存取單體式應用程式

執行下列指令，找出單體應用程式的外部 IP 位址。

```
kubectl get service monolith
```

輸出結果應該會類似下列內容：

NAME	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
monolith	10.3.251.122	203.0.113.0	80:30877/TCP	3d

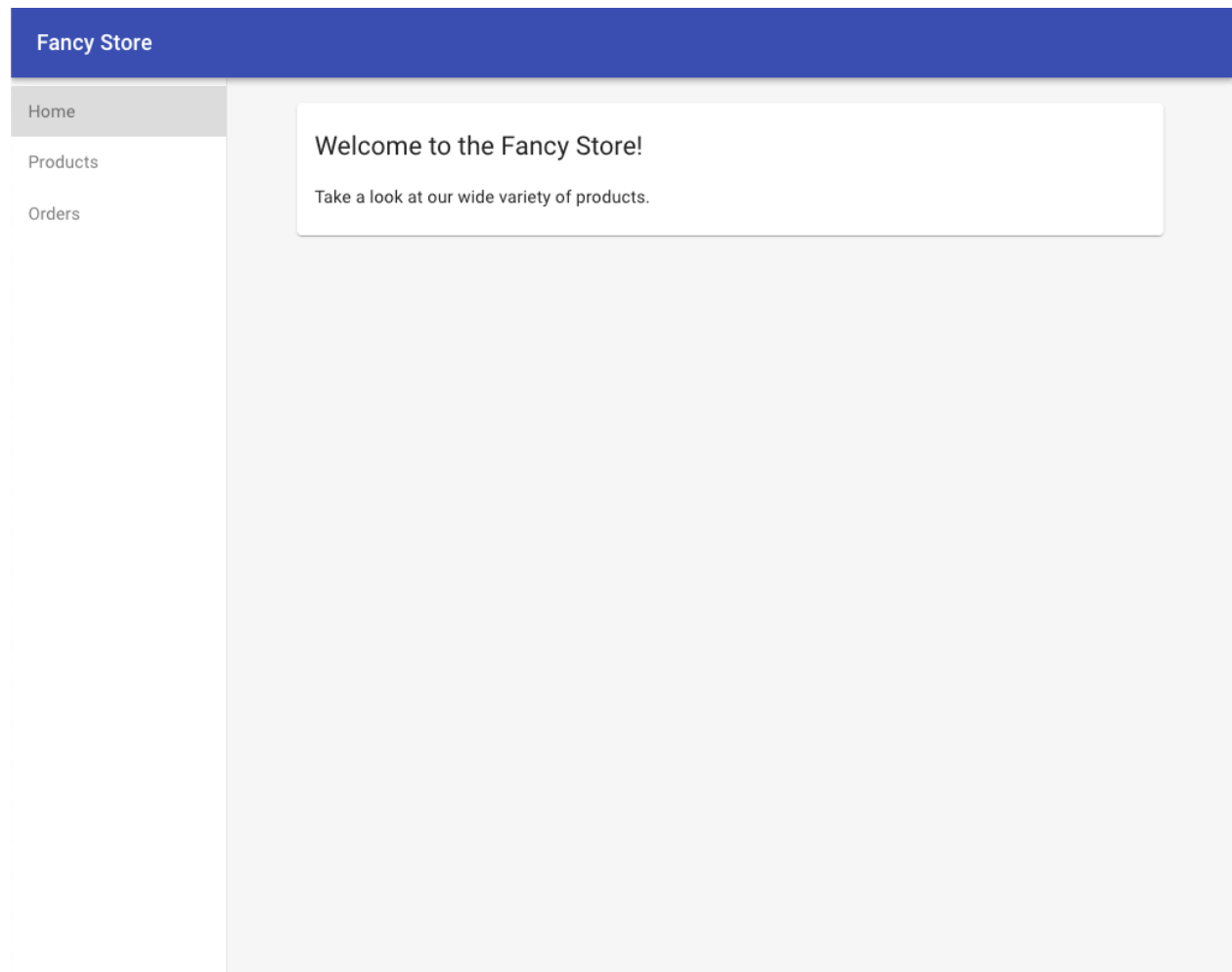
注意：系統必須為此佈建外部負載平衡器和 IP，因此需要一些時間。如果輸出內容將外部 IP 列為

<pending> 請稍候片刻，然後再試一次。

確認單體的外部 IP 位址後，請複製這個 IP 位址。將瀏覽器指向這個網址 (如 <http://203.0.113.0>)，以檢查您的單體是否能存取。

NOTE

請記住這個 IP 位址，後續步驟會繼續使用。您隨時都能透過相同指令再次找到這個檔案。



您應該會看到單體式網站的歡迎頁面，如上圖所示。歡迎頁面是靜態頁面，稍後會由前端微服務提供。現在，您的單體應用程式已在 Kubernetes 上完整運作！

6. 將訂單遷移至微服務

既然我們已在 GKE 上執行現有的單體式網站，就開始將各項服務拆分成微服務吧。規劃時通常會著重於將哪些服務拆分成較小的部分，一般是根據應用程式的特定部分 (例如業務領域)。為了示範，我們建立了一個簡單的範例，並根據業務領域將每個服務分開：訂單、產品和前端。程式碼已遷移完畢，因此我們將專心在 Google Kubernetes Engine (GKE) 上建構及部署服務。

建立新的訂單微服務

首先要分出的服務是訂單服務。我們會使用提供的獨立程式碼集，為這項服務建立獨立的 Docker 容器。

使用 Google Cloud Build 建立 Docker 容器

由於我們已為您遷移程式碼集，第一步是使用 Google Cloud Build 建立訂單服務的 Docker 容器。

一般來說，您必須採取兩個步驟：建構 Docker 容器，然後將容器推送至存放區，以便儲存映像檔供 GKE 提取。但我們可以讓生活更輕鬆，只要一個指令，就能使用 Google Cloud Build 建構 Docker 容器，並將映像檔放入 Google Cloud Container Registry！這樣一來，我們就能發出單一指令，建構映像檔並移至容器登錄檔。如要查看手動建立 Docker 檔案並推送的程序，請參閱[這篇文章](#)。

Google Cloud Build 會壓縮目錄中的檔案，並將檔案移至 Google Cloud Storage bucket。建構程序會從 bucket 擷取所有檔案，並使用 Dockerfile 執行 Docker 建構程序。我們在 Docker 映像檔中指定了 `--tag` 旗標，並將主機設為 `gcr.io`，產生的 Docker 映像檔會推送至 Google Cloud Container Registry。

執行下列指令，建構 Docker 容器並推送至 Google Container Registry：

```
cd ~/monolith-to-microservices/microservices/src/orders
gcloud builds submit --tag gcr.io/${GOOGLE_CLOUD_PROJECT}/orders:1.0.0 .
```

這個程序需要幾分鐘，完成後終端機中會顯示類似以下的輸出內容：

```
-----
-----
-----
ID                                CREATE_TIME                        DURATION  SOURCE
IMAGES                            STATUS
1ae295d9-63cb-482c-959b-bc52e9644d53  2019-08-29T01:56:35+00:00  33S
gs://<PROJECT_ID>_cloudbuild/source/1567043793.94-
abfd382011724422bf49af1558b894aa.tgz  gcr.io/<PROJECT_ID>/orders:1.0.0  SUCCESS
```

如要查看建構記錄或即時監看程序，請前往 Google Cloud 控制台。按一下左上角的選單按鈕，然後向下捲動至「工具」→「Cloud Build」，並按一下「記錄」。這裡會列出您先前建立的所有版本，其中應該只有您剛建立的版本。

Build History						
Filter builds						
Build	Source	Git commit	Trigger	Started	Image target	Image tag
985ce3e5-afba...	gs://gcb-docs-project_cloudbuild/source/1508159187.8-b0d8841d51674a30aebd1e55bb99486f.tgz	—	—	9:06 AM	gcr.io/gcb-docs-project/quickstart-image	latest
d2e89f1b-cbf9...	gs://gcb-docs-project_cloudbuild/source/1508158566.55-725755714baa4b7e9e99984c422ec4e2.tgz	—	—	8:56 AM	gcr.io/gcb-docs-project/quickstart-image	latest

點選建構作業 ID，即可查看該建構作業的所有詳細資料，包括記錄輸出內容。

在「建構作業詳細資料」頁面，點選「建構資訊」部分中的映像檔名稱，即可查看建立的容器映像檔。

Status	Build successful
Build id	fbcd08-44bc-46e8-bd18-ba3465ade0cb
Image	gcr.io/gcb-docs-project/quickstart-image
Source	gs://gcb-docs-project_cloudbuild/source/1534173107.74-8167ff55ae2e4b8ca34779daf3090eed.tgz
Started	August 13, 2018 at 11:11:49 AM UTC-4
Build time	15 sec

Build steps

gcr.io/cloud-builders/docker build -t gcr.io/gcb-docs-project/quickstart-image.	0.7 sec
---	---------

將容器部署至 GKE

我們已將網站容器化，並將容器推送至 Google Container Registry，現在該部署至 Kubernetes 了！

Kubernetes 會以 [Pod](#) 的形式呈現應用程式；Pod 是容器/緊耦合容器群組的單位，也是 Kubernetes 中最小的可部署單位。在本教學課程中，每個 Pod 只包含微服務容器。

如要在 GKE 叢集上部署和管理應用程式，您必須與 Kubernetes 叢集管理系統進行通訊。一般來說，此操作會透過 Cloud Shell 內的 **kubectl** 指令列工具進行。

首先，我們會建立 [Deployment](#) 資源。Deployment 會管理應用程式的多個副本，並將這些副本安排在叢集的各個節點上執行。就這個範例而言，部署作業只會執行應用程式的一個 Pod。Deployment 會建立 [ReplicaSet](#)，確保 Pod 數量符合預期。ReplicaSet 負責確保指定數量的副本隨時都在執行。

下方的 `kubectl create deployment` 指令會讓 Kubernetes 在叢集上建立名為 **orders** 的 Deployment，並使用 **1** 個副本。

執行下列指令來部署應用程式：

```
kubectl create deployment orders --image=gcr.io/${GOOGLE_CLOUD_PROJECT}/orders:1.0.0
```

注意事項

最佳做法是使用 YAML 檔案宣告 Kubernetes 叢集的變更 (例如建立或修改部署作業或服務)，並使用 GitHub 或 Cloud Source Repositories 等原始碼控管系統儲存這些變更。

詳情請參閱下列資源：

[Kubernetes Deployment](#)

驗證部署作業

執行下列指令，確認 Deployment 已順利建立。Pod 狀態可能需要幾分鐘才會變成「Running」：

```
kubectl get all
```

輸出內容：

NAME	READY	STATUS	RESTARTS	AGE
pod/monolith-779c8d95f5-dxnz1	1/1	Running	0	15h
pod/orders-5bc6969d76-kdxkk	1/1	Running	0	21s

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/kubernetes	ClusterIP	10.39.240.1	<none>	443/TCP	19d
service/monolith	LoadBalancer	10.39.241.130	34.74.209.57	80:30412/TCP	15h

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/monolith	1/1	1	1	15h
deployment.apps/orders	1/1	1	1	21s

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/monolith-779c8d95f5	1	1	1	15h
replicaset.apps/orders-5bc6969d76	1	1	1	21s

這個輸出內容顯示了幾件事。您會看到目前的 Deployment、所需 Pod 數量為 1 的 ReplicaSet，以及正在執行的 Pod。看來一切都順利建立完成！

實用提示

您也可以透過控制台查看 Kubernetes 部署作業，方法是前往左上方的選單，依序前往 Kubernetes Engine → Workloads

公開 GKE 容器

我們已在 GKE 上部署應用程式，但無法從叢集外部存取。根據預設，您在 GKE 上執行的容器無法從網際網路存取，因為這些容器沒有外部 IP 位址。您必須透過 [Service](#) 資源，明確將應用程式公開至網際網路傳出的流量。Service 可為應用程式的 Pod 提供網路和 IP 支援。GKE 會為應用程式建立外部 IP 和負載平衡器 ([需要計費](#))。

注意事項

為配合本示範的學習目標，我們簡化了服務公開作業。平常應該是使用 API 閘道來保護公開端點。如要進一步瞭解微服務最佳做法，請參閱[這篇文章](#)。

部署訂單服務時，我們透過 Kubernetes Deployment 在內部公開了通訊埠 8081。如要對外公開這項服務，我們需要建立 LoadBalancer 類型的 Kubernetes 服務，將外部通訊埠 80 的流量轉送至訂單服務的內部通訊埠 8081。執行下列指令，將網站公開至網際網路：

```
kubectl expose deployment orders --type=LoadBalancer --port 80 --target-port 8081
```

存取本服務

GKE 會將外部 IP 位址指派給 Service 資源，而非 Deployment。如要找出 GKE 為應用程式佈建的外部 IP，請透過 `kubectl get service` 指令檢查該 Service：

```
kubectl get service orders
```

輸出內容：

NAME	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
orders	10.3.251.122	203.0.113.0	80:30877/TCP	3d

確認應用程式的外部 IP 位址後，請複製這個 IP 位址。請儲存這個網址，在後續步驟中，將單體應用程式指向新的訂單服務！

重新設定單體應用程式

由於我們已從單體應用程式中移除訂單服務，因此必須修改單體應用程式，以指向新的外部訂單微服務。

分解單體式應用程式時，我們會將單一程式碼集中的程式碼片段移至多個微服務，並分別部署。由於微服務在不同的伺服器上執行，我們無法再將服務網址當做絕對路徑參照，必須改為將流量導向訂單微服務的新伺服器位址。請注意，這會導致單體服務停機，以更新每個已拆分服務的網址。在微服務遷移過程中，規劃將微服務和單體應用程式移至正式環境時，應將這點納入考量。

我們需要更新單體應用程式中的設定檔，以指向新的訂單微服務 IP 位址。使用 nano 編輯器，將本機網址換成新訂單微服務的 IP 位址。執行下列指令來編輯

```
cd ~/monolith-to-microservices/react-app
nano .env.monolith
```

開啟編輯器後，檔案應如下所示：

```
REACT_APP_ORDERS_URL=/service/orders
REACT_APP_PRODUCTS_URL=/service/products
```

將 `REACT_APP_ORDERS_URL` 替換為新格式，並將訂單微服務 IP 位址替換為以下內容：

```
REACT_APP_ORDERS_URL=http://<ORDERS_IP_ADDRESS>/api/orders
REACT_APP_PRODUCTS_URL=/service/products
```

依序按下 `CTRL+O` 鍵、`ENTER` 鍵和 `CTRL+X` 鍵，在 nano 編輯器中儲存檔案。

前往您剛在這個檔案中設定的網址，測試新的微服務。網頁應傳回訂單微服務的 JSON 回應。

接著，我們需要重建單體式前端，並重複建構程序，為單體式應用程式建構容器，然後重新部署至 GKE 叢集。執行下列指令來完成這些步驟：

重建單體設定檔

```
npm run build:monolith
```

使用 Google Cloud Build 建立 Docker 容器

```
cd ~/monolith-to-microservices/monolith
gcloud builds submit --tag gcr.io/${GOOGLE_CLOUD_PROJECT}/monolith:2.0.0 .
```

將容器部署至 GKE

```
kubectl set image deployment/monolith
monolith=gcr.io/${GOOGLE_CLOUD_PROJECT}/monolith:2.0.0
```

前往瀏覽器中的單體應用程式，然後前往「訂單」頁面，確認應用程式現在會連至新的訂單微服務。所有訂單 ID 都應以 -MICROSERVICE 為後綴，如下所示：

Orders			
Order Id	Date	Total Items	Cost
ORD-000001-MICROSERVICE	7/01/2019	1	\$67.99
ORD-000002-MICROSERVICE	7/24/2019	1	\$124
ORD-000003-MICROSERVICE	8/03/2019	1	\$12.49
ORD-000004-MICROSERVICE	8/14/2019	2	\$89.83
ORD-000005-MICROSERVICE	8/29/2019	1	\$12.3

步驟 7 · 約 10 分鐘

7. 將產品遷移至微服務

建立新的產品微服務

接著遷移產品服務，繼續將服務拆分成多個部分。我們會按照與上一個步驟相同的程序操作。執行下列指令，建構 Docker 容器、部署容器，並透過 Kubernetes 服務公開容器。

使用 Google Cloud Build 建立 Docker 容器

```
cd ~/monolith-to-microservices/microservices/src/products
gcloud builds submit --tag gcr.io/${GOOGLE_CLOUD_PROJECT}/products:1.0.0 .
```

將容器部署至 GKE

```
kubectl create deployment products --
image=gcr.io/${GOOGLE_CLOUD_PROJECT}/products:1.0.0
```

公開 GKE 容器

```
kubectl expose deployment products --type=LoadBalancer --port 80 --target-port 8082
```

按照與訂單服務相同的方式，使用下列指令找出產品服務的公開 IP：

```
kubectl get service products
```

輸出內容：

NAME	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
products	10.3.251.122	203.0.113.0	80:30877/TCP	3d

請儲存這個 IP 位址，在後續步驟中，重新設定單體應用程式，以指向新的 Products 微服務。

重新設定單體應用程式

使用 nano 編輯器，將本機網址換成新 Products 微服務的 IP 位址：

```
cd ~/monolith-to-microservices/react-app
nano .env.monolith
```

開啟編輯器後，檔案應如下所示：

```
REACT_APP_ORDERS_URL=http://<ORDERS_IP_ADDRESS>/api/orders
REACT_APP_PRODUCTS_URL=/service/products
```

將 `REACT_APP_PRODUCTS_URL` 替換為新格式，並將產品微服務的 IP 位址替換為以下內容：

```
REACT_APP_ORDERS_URL=http://<ORDERS_IP_ADDRESS>/api/orders
REACT_APP_PRODUCTS_URL=http://<PRODUCTS_IP_ADDRESS>/api/products
```

依序按下 `CTRL+O` 鍵、`ENTER` 鍵和 `CTRL+X` 鍵，在 nano 編輯器中儲存檔案。

前往您剛在這個檔案中設定的網址，測試新的微服務。網頁應傳回產品微服務的 JSON 回應。

接著，我們需要重建單體式前端，並重複建構程序，為單體式應用程式建構容器，然後重新部署至 GKE 叢集。執行下列指令來完成這些步驟：

重建單體設定檔

```
npm run build:monolith
```

使用 Google Cloud Build 建立 Docker 容器

```
cd ~/monolith-to-microservices/monolith
gcloud builds submit --tag gcr.io/${GOOGLE_CLOUD_PROJECT}/monolith:3.0.0 .
```

將容器部署至 GKE

```
kubectl set image deployment/monolith
monolith=gcr.io/${GOOGLE_CLOUD_PROJECT}/monolith:3.0.0
```

前往瀏覽器中的單體應用程式，然後前往「產品」頁面，確認應用程式現在會連至新的產品微服務。所有產品名稱都應加上 MS- 前置字元，如下所示：



MS - Vintage Typewriter - \$67.99



MS - Vintage Camera Lens - \$12.49



MS - Home Barista Kit - \$124



MS - Terrarium - \$36.45



MS - Film Camera - \$2245



MS - Vintage Record Player - \$65.5



MS - Metal Camping Mug - \$24.33



MS - City Bike - \$789.5



MS - Air Plant - \$12.3

8. 將前端遷移至微服務

遷移程序的最後一步，是將前端程式碼移至微服務，並關閉單體應用程式！完成這個步驟後，我們就成功將單體應用程式遷移至微服務架構了！

建立新的前端微服務

按照前兩個步驟的相同程序，建立新的前端微服務。

先前重建單體應用程式時，我們更新了設定，將其指向單體應用程式，但現在需要對前端微服務使用相同的設定。執行下列指令，將微服務網址設定檔複製到前端微服務程式碼集：

```
cd ~/monolith-to-microservices/react-app
cp .env.monolith .env
npm run build
```

完成後，請按照與先前步驟相同的程序操作。執行下列指令，建構 Docker 容器、部署容器，並透過 Kubernetes 服務公開容器。

使用 Google Cloud Build 建立 Docker 容器

```
cd ~/monolith-to-microservices/microservices/src/frontend
gcloud builds submit --tag gcr.io/${GOOGLE_CLOUD_PROJECT}/frontend:1.0.0 .
```

將容器部署至 GKE

```
kubectl create deployment frontend --
image=gcr.io/${GOOGLE_CLOUD_PROJECT}/frontend:1.0.0
```

公開 GKE 容器

```
kubectl expose deployment frontend --type=LoadBalancer --port 80 --target-port 8080
```

刪除單體應用程式

現在所有服務都以微服務的形式運作了，請刪除單體應用程式！請注意，在實際遷移作業時，您也需要變更 DNS 等設定，讓現有的網域名稱指向應用程式的新前端微服務。執行下列指令，刪除單體應用程式：

```
kubectl delete deployment monolith
kubectl delete service monolith
```

測試作品

如要確認一切正常運作，單體服務的舊 IP 位址應該無法運作，而前端服務的新 IP 位址應會代管新應用程式。如要查看所有服務和 IP 位址的清單，請使用下列指令：

```
kubectl get services
```

輸出內容應類似以下所示：

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
frontend	LoadBalancer	10.39.246.135	35.227.21.154	80:32663/TCP	12m
kubernetes	ClusterIP	10.39.240.1	<none>	443/TCP	18d
orders	LoadBalancer	10.39.243.42	35.243.173.255	80:32714/TCP	31m
products	LoadBalancer	10.39.250.16	35.243.180.23	80:32335/TCP	21m

確認前端微服務的外部 IP 位址後，請複製這個 IP 位址。將瀏覽器指向這個網址 (如 <http://203.0.113.0>)，以檢查您的前端是否能存取。網站應與將單體式應用程式拆解為微服務前相同！

9. 清除所用資源

準備就緒後，如要清除所有活動，最簡單的方法就是刪除專案。刪除專案會一併刪除本程式碼研究室建立的所有資源，確保不會產生非預期的週期性費用。在 Cloud Shell 中執行下列指令，其中 `PROJECT_ID` 是完整的專案 ID，而非專案名稱。

```
gcloud projects delete [PROJECT_ID]
```

系統提示時，輸入「Y」確認刪除。

如果不確定專案 ID，可以從 Cloud Shell 執行 `gcloud config list | grep project`，取得目前的專案 ID。

10. 恭喜！

您已成功將單體式應用程式拆分為微服務，並部署至 Google Kubernetes Engine！

後續步驟

如要進一步瞭解 Kubernetes，請參閱下列程式碼研究室：

- 在 [Google Kubernetes Engine](#) 上部署、擴充及更新網站
- 在 [Kubernetes](#) 上使用 Node.js 建構 Slack 機器人
- 使用 [Spinnaker](#) 持續將軟體更新推送至 [Kubernetes](#)
- 將 [Java](#) 應用程式部署至 [Google Kubernetes Engine](#) 上的 [Kubernetes](#)

其他資源

- Docker - <https://docs.docker.com/>
 - Kubernetes - <https://kubernetes.io/docs/home/>
 - Google Kubernetes Engine (GKE) - <https://cloud.google.com/kubernetes-engine/docs/>
 - Google Cloud Build - <https://cloud.google.com/cloud-build/docs/>
 - Google Container Registry - <https://cloud.google.com/container-registry/docs/>
 - 將單體式應用程式遷移至微服務 - <https://cloud.google.com/solutions/migrating-a-monolithic-app-to-microservices-gke>
-